

Article

Building Natural Language Interfaces Using Natural Language Understanding and Generation: A Case Study on Human–Machine Interaction in Agriculture

Yongheng Zhang ¹, Siyi Yao ¹, Peng Wang ¹, Hao Wu ², Zhipeng Xu ¹, Yongmei Wang ^{1,3,*}
and Youhua Zhang ^{1,3,*}

¹ School of Information and Computer Science, Anhui Agricultural University, Hefei 230036, China

² School of Computer Science and Technology, University of Science and Technology of China, Hefei 230027, China

³ Anhui Provincial Engineering Laboratory for Beidou Precision Agriculture Information, Hefei 230036, China

* Correspondence: wym0152@foxmail.com (Y.W.); zhangyh@ahau.edu.cn (Y.Z.)

Abstract: The human–machine interaction of existing agricultural measurement and control platforms lacks user-friendliness and requires manual operation by trained professionals. The recent development of natural language processing technology may bring some interesting changes. We propose a pipeline for building a natural language human–machine interaction interface to provide a better interaction for agricultural measurement and control platforms. Our construction process uses a new method of collecting training data based on the dynamic tuple language framework to synthesize natural language commands entered by the user into structured AOM statements (Action-Object-Member). To construct a mapping of the human–machine interface from natural language commands to AOM invocations, we propose an end-to-end framework that uses a special mask mechanism to improve the BERT-based Seq2Seq model to capture global sequence relations. Experimental results of data collection methods and NL2AOM demonstrate that our pipeline has good performance and a reasonable response time. Finally, we developed desktop and mobile platform applications based on the proposed model and used them in real agricultural scenarios.

Keywords: human–machine interaction; natural language processing; semantic parsing; agricultural measurement and control; natural user interface



Citation: Zhang, Y.; Yao, S.; Wang, P.; Wu, H.; Xu, Z.; Wang, Y.; Zhang, Y. Building Natural Language Interfaces Using Natural Language Understanding and Generation: A Case Study on Human–Machine Interaction in Agriculture. *Appl. Sci.* **2022**, *12*, 11830. <https://doi.org/10.3390/app122211830>

Academic Editor: João M. F. Rodrigues

Received: 22 October 2022

Accepted: 17 November 2022

Published: 21 November 2022

Publisher's Note: MDPI stays neutral with regard to jurisdictional claims in published maps and institutional affiliations.



Copyright: © 2022 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Human–machine interaction [1] has attracted the attention of many researchers, and with the development of artificial intelligence (AI) technology and Internet of Things (IoT) technology, human–machine interaction is gradually being applied in the field of agriculture. The purpose of this technology is to achieve real-time information exchange between humans and machines, where workers can operate equipment remotely as well as query data in real time and return the results simply by voice or text input. Almost all existing human–machine interaction in agriculture is based on Graph User Interface (GUI) [2]. Staff with domain-specific knowledge are required to perform limited operations through the GUI, such as the deployment and commissioning of agricultural field equipment with the click of a button. However, if the control commands are too cumbersome, the time cost is too high, and the program of operation is fixed, the human–machine interaction is not friendly.

The construction of a natural user interface (NUI) for agricultural measurement and control [3] is an effective means of solving this problem. NUI means that the user can interact with the machine naturally and directly, reducing the complexity of the interaction. In the web application domain [4,5]; for example, web services hosted in the cloud (containing information queries on weather, sports, life, etc.) provide relevant data and services to users

through web program interfaces. In the mobile domain, relevant natural user interaction interfaces are also provided, with similar applications such as Apple's Siri [6], Microsoft's Cortana [7], and virtual assistants such as Huawei's Xiao E. In the database field, building natural language interfaces for databases [8] has also become a research hotspot; natural language interface of database (NLIDB) is a product that combines database technology and natural language-understanding technology. It aims to provide a natural language interface for various operations related to databases so that users can make query requests clearly and concisely.

The usefulness of a virtual assistant depends heavily on its breadth, i.e., the number of services it supports [9]. The field-based human-machine interaction related to natural language interaction, such as agricultural measurement and control, is focused on a particular service. It needs to be optimized for this service to improve its accuracy and avoid damage to equipment due to misuse. In related studies, keyword-based interactions are the most popular. Austerjost et al. [10] built a virtual artificial intelligence assistant for the lab. The assistant looks for key phrases in voice commands that match a pre-built dictionary of device actions, but this approach is less targeted. Mahnoosh et al. [11] constructed a virtual AI assistant for smart home environments, extracting linguistic elements such as location, environment, and movement from voice commands in a way that is somewhat goal-oriented. Noura et al. [12] proposed the VISH grammar, which uses multiple rules to extract multiple types of entities from voice commands for analyzing the intent in voice commands and controlling target devices in smart home environments. A more targeted model is needed for applications of agricultural measurement and control systems.

The core task of the natural user interface for the agricultural domain studied in this paper is to convert natural language commands entered by the user via speech or text into a machine language that can be executed by a computer system. The core problems are as follows:

- How to construct a high-quality human-machine interaction dataset about the agricultural domain, i.e., a reasonable linguistic framework is needed to translate between the natural language commands entered by the user and the machine commands that can be understood by the computer system. An intermediate language framework needs to be found to structure the natural language for storage.
- How to build a natural user interface with scientific generalization capability and extensibility, and effectively complete the mapping between natural language command sequences and computer system call interfaces.

The first step, the natural language understanding (NLU) problem in natural language processing [13], is to solve the essential problem of natural user interfaces. It usually requires the analysis of commands at the semantic level, including text partitioning, lexical annotation, keyword extraction, similarity calculation, and dependent syntactic analysis [14], among others. Understanding the actual intent of natural language instructions completes key information extraction to populate a specific intermediate language framework. Regarding the second step, the main focus is on how to implement sequence-to-sequence conversions that map to corresponding parameters, making the natural user interface strongly generalizable as well as extensible, the effect of which depends heavily on the first step.

Regarding the research on language storage, the knowledge graph is one of the current research hotspots, among which RDF, RDFS, and OWL are common ways for storing data in the knowledge graph; all are metadata written based on XML and widely used in computer transfer data for data models of machine understanding. Dar et al. [15] designed a data model based on this framework for transforming natural language interrogatives into structured queries. Starting from the syntactic structure of natural language questions, the method of identifying entities and relations, and using the corpus to build a mapping from entities to the knowledge base, disambiguating predicates, and then transforming them into a computer-understandable structured query language. Jin et al. [16] also proposed a semantic expansion-based approach to identify entities and relations in questions

based on RDF triadic data storage, construct query graphs for natural language questions, narrow the search space of the knowledge base, and implement a semantic matching-based knowledge base Q&A. Ait-Mlouk et al. [17] developed an architecture to provide an interactive user interface that uses machine learning methods of intent classification and natural language understanding to understand user intent and generate SPARQL queries to build a natural language understanding chatbot for linked data based on knowledge graphs. Song et al. [18] proposed a semantic element combination-based Chinese problem understanding method for the natural language problem intent understanding piece, using vocabulary and rule recognition to extract the semantic elements of the problem, combined with dependent syntactic analysis to generate a normalized semantic expression to structure the knowledge and to effectively understand the Chinese problem's semantics. Xu et al. [19] proposed an efficient pipeline framework to model the user's query intent as a phrase-level dependency that is instantiated based on a specific knowledge base to construct the final structured query. E.T.-H et al. [20] provide a natural human-machine interaction interface for the medical domain that combines a hybrid approach of a cosine similarity and word frequency-inverse document matrix to determine user intent, and the results are returned to the user through an interface of lines. Xu et al. [8] proposed a database-oriented natural language interface, proposed a reliance on sketches to accomplish the representation of natural language instructions, and also proposed a sequence-to-set model and a column attention mechanism to synthesize sketch-based SQL query statements. Tian et al. [21] proposed a field-embedding method based on the content of data tables based on SQLNet, which uses the content stored in each field in the data table to embed the field representation, and proposed a data enhancement method. The natural language is transformed into a SQL query statement that the database can directly execute and parse, effectively solving the problem of intelligent interaction and human-machine dialogue. Su et al. [9] constructed a natural language interface for a web application programming interface and proposed an end-to-end framework to build NL2API for a specific web application programming interface, providing a crowdsourced data process via a hierarchical probabilistic model to filter high-quality datasets, and constructed an end-to-end framework for NL2API. VS et al. [22] proposed a framework for discovering composable service sets based on complex user requirements, using natural language processing and semantic-based techniques to extract functional semantics from service datasets. Qiu et al. [23] proposed AliMe chat, which integrates information retrieval and Seq2Seq for generating joint model results. Wu et al. [24] constructed a natural language-integrated query and control interface for distributed SCADA systems using hierarchical semantic parsing algorithms to solve the complex natural language interface problem for data query and real-time control.

In summary, the design of a natural user interface in agriculture is important to reduce operation and maintenance costs and improve the efficiency of agricultural measurement and control systems. The purpose of our work is to design a natural user interface for the agricultural field. However, before our work can be carried out, we are faced with a number of problems waiting to be solved. As with most natural language processing work, there is a lack of training sets with high-quality labels to drive the training of models [25]; intelligent computational models with strong scientific generalization capability and high scalability are needed to build natural user interfaces to solve natural language interaction problems in various complex agricultural environments. In the design of natural user interfaces in the agricultural domain, there is a need to combine natural language understanding, natural language generation, and deep learning theories to develop a scientific, rational, and efficient natural user interface for agriculture. The contributions of this paper are shown below.

- We have proposed a new language storage framework based on dynamic tuple extraction, and accordingly define a structured intermediate language AOM (Action-Object-Member), which is mapped to corresponding structured intermediate statements

according to the complexity of natural language commands, respectively. It can collect high-quality training data efficiently.

- We have built an end-to-end framework from natural language to AOM (NL2AOM). We have used an improved Seq2Seq model based on BERT using a special mask mechanism to implement a sequence-to-sequence concept based on self-attention to obtain global information about natural language commands.
- We have collected natural language command data from real agricultural environments to train and test our model and applied it to real agricultural environments. Our experiments show that the proposed human–machine user interface in agriculture can better solve the natural human–machine interaction problem and achieves good results in the evaluation of performance metrics, with response times that meet the requirements of human–machine interactions. A robust baseline is provided for the field.

2. Proposed System

The workflow of human–machine interaction abstraction for agriculture proposed in this paper is shown in Figure 1, which aims to provide a friendly human–machine interaction for agricultural measurement and control systems. Step 1 describes the user inputting natural language commands by voice or text; Step 2 describes the process of unstructured natural language to structured intermediate language AOM; Step 3 describes the mapping of extracted key indicator information into the corresponding parameters to send to Raspberry Pi; Steps 4 and 5 describe the process of controlling field devices and debugging sensors, respectively. Step 6 describes the process of sending the data measured by the sensor back to the user. The proposed approach in this paper implements the transformation of a discrete, unstructured natural language into a structured intermediate language. We illustrate this process in terms of parameter passing. We enter the natural language command “请帮我先打开大棚的空调，然后关闭稻田的喷头。” (“Please help me turn on the air conditioner in the greenhouse and then turn off the sprinkler in the rice field.”), after structuring, a structured AOM statement is generated: [{Action_1: “打开” (“turn on”), Object_1: “大棚” (“greenhouse”), Member_1: “空调” (“air conditioner”)}, {Action_1: “关闭” (“turn off”), Object_1: “稻田” (“rice field”), Member_1: “喷头” (“sprinkler”)}]. The keywords in the generated AOM statements are then mapped to the corresponding parameters and sent to the Raspberry Pi in the field, and the Raspberry Pi port recognizes the corresponding information to achieve the corresponding manipulation functions.

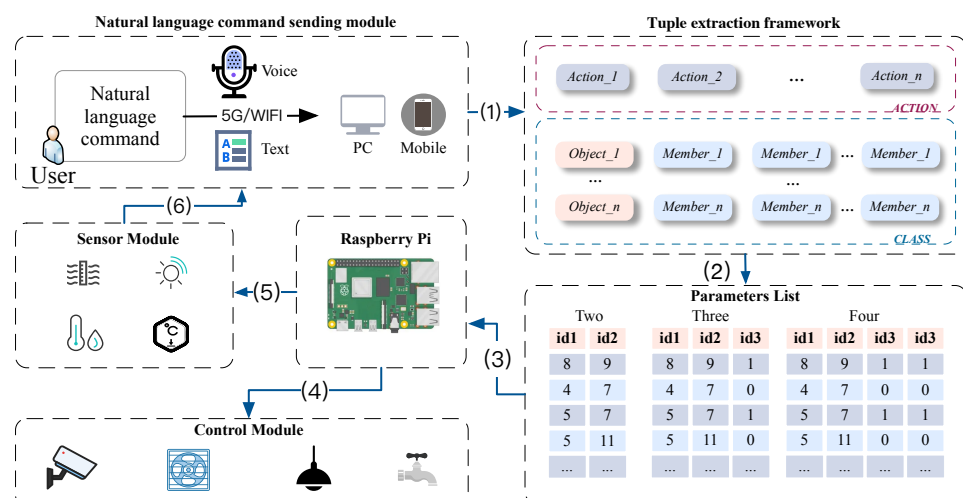


Figure 1. Human–machine interaction abstraction workflow in agriculture.

3. Training Data Collection

In this section, we propose a dynamic tuple extraction-based language framework for NL2AOM to collect training data. The basic idea is to use a sketch of a dynamic language tuple extraction framework that is highly consistent with the data structure of AOM. Only the slots in the sketch need to be filled to complete the semantic parsing of natural language so that natural language commands can be automatically generated into AOM statements. The sketch is designed to be general and reasonable enough so that all-natural language commands can be structured and extracted through the sketch. Thus, the use of sketches does not hinder the scientific generalization ability and universality of our approach. We will introduce the design philosophy and details of sketching in Section 3.1. The sketch is designed to provide a new approach to structured extraction for discrete natural languages, taking into account the global language and the semantic relationships between contexts. Section 3.2 describes, in detail, the technical details of implementing a dynamic language tuple extraction framework, explaining the proposed algorithm and giving the associated pseudo-code implementation.

3.1. Sketch-Based Synthesis of AOM Statements

Our dynamic tuple extraction-based language storage framework is shown in Figure 2, and this subsection describes its design concept in detail. The sketch consists of two parts, one is the ACTION module, and the other is the CLASS module, where the CLASS module includes two types of attributes; Object and Member attributes. Considering the actual production work in the agricultural field, most of the commands are related to the manipulation type, such as turning on a field node device, checking the temperature and humidity of a node, and so on. These instructions contain action keywords, so when structuring and parsing the instructions, we can consider extracting the verbs into a separate module as the action attributes in the structured instructions. Next, we define a CLASS module, which can be considered a “class” containing both object and member attributes; for example, the instruction “打开一下大棚的吊灯” (“turn on the chandelier of the shed”), which can consider “大棚” (“shed”) as an object property, while “吊灯” (“chandelier”) is a member property, and then synthesize the AOM statements in the order {Action_1: “打开” (“turn on”), Object_1: “大棚” (“shed”), Member_1: “吊灯” (“chandelier”)}. However, the natural language structuring is not necessarily in the form of a fixed triple; for example, the instruction “分别打开大棚和稻田里面的传感器” (“turn on the sensors inside the shed and the rice field respectively”), where the action attribute is considered “打开” (“turn on”), “大棚” (“shed”) is Object_1, “稻田” (“paddy field”) is Object_2, and “传感器” (“sensor”) is Member_1, which translates into an AOM statement as {Action_1: “打开” (“turn on”), Object: {Object_1: “大棚” (“shed”), Object_2: “稻田” (“paddy field”)}, Member_1: “传感器” (“sensor”)}. In other words, from the sketching point of view, we first fill “打开” (“turn on”) into Action_1, then find two object properties for the parallel relationship, and fill the key indicators “大棚” (“shed”) and “稻田” (“Paddy”) into the corresponding slots. Finally, both objects have the same type of member, i.e., both object slots point to the same member, Member_1: “传感器” (“sensor”). Thus, the structured extraction of natural language is completed. A sample example is shown in Figure 2.

Therefore, we next summarize the common types of natural language instructions formally. In this paper, we define natural language commands as basic natural language commands, complex natural language commands, object juxtaposition natural language commands, member juxtaposition natural language commands, and object nested natural language commands in conjunction with actual agricultural scenarios. The specific explanations are as follows.

- Basic natural language commands (a): Natural language commands that do not contain any nested, parallel, or other complex relationships, such as “请帮我打开大棚的吊灯” (“Please help me turn on the chandelier in the shed”), the generated AOM statement is {Action_1: “打开” (“turn on”), Object_1: “大棚” (“shed”), Member_1: “吊灯” (“chandelier”)}, in the form of a single keyword for each attribute.

- Complex natural language commands(b): Multiple simple natural language commands are combined. If the basic natural language command is S_1 , then the complex natural language command is $(S_1, S_2, S_3, \dots, S_n)$, where n is the number of simple natural language commands.
- Object juxtaposition natural language commands (c): As the name suggests, two objects point to the same member, e.g., the command “请帮我同时打开稻田和麦田的喷洒机” (“Please help me turn on the sprinklers in both the rice field and the wheat field”), the keywords “稻田” (“rice field”) and “麦田” (“wheat field”) are Object_1 and Object_2, respectively, both pointing to the same type of member property, i.e., Member_1: “喷洒机” (“sprinkler”). Theoretically generated AOM statements are {Action_1: “打开” (“turn on”), Object: {Object_1: “稻田” (“rice field”), Object_2: “麦田” (“wheat field”)}, Member_1: “喷洒机” (“sprinkler”)}.
- Member parallel natural language commands (d): This type of command manipulates member variables that exist side by side; for example, the command “分别打开稻田的喷头和传感器” (“Turn on the nozzle and sensor of the paddy field respectively”), in which the object properties are “喷头” (“nozzle”) and “传感器” (“sensor”), respectively, is a parallel relationship, and the common object they belong to is “稻田” (“paddy field”), i.e., Object_1 points to both Member_1 and Member_2. The generated AOM statement is {Action_1: “打开” (“turn on”), Object_1: “稻田” (“paddy field”), Member: {Member_1: “喷头” (“nozzle”), Member_2: “传感器” (“sensor”)}}.
- Object nested natural language commands (e): This type of command is relatively complex; for example, “请帮我查看一下稻田里面土壤的温湿度情况” (“Please help me check the temperature and humidity of the soil in the rice field”). Here, “稻田” (“rice field”) and “土壤” (“soil”) are both object attributes, but “土壤” (“soil”) is nested inside “稻田” (“rice field”); that is, “温湿度” (“temperature and humidity”) is a member of “土壤” (“soil”), not a member of “稻田” (“rice field”), as in Figure 2, Object_1 points to the line; that is, on behalf of Object_1 it tries to find its net nested object Object_i ($i = 2, 3, \dots, n$). Then the final object points to the member property in the directive. Then the generated AOM statement is {Action_1: “查看” (“check”), Object_1: “稻田” (“rice field”), Object_2: “土壤” (“soil”), Member: {Member_1: “温度” (“temperature”): Member_2: “湿度” (“humidity”)}}

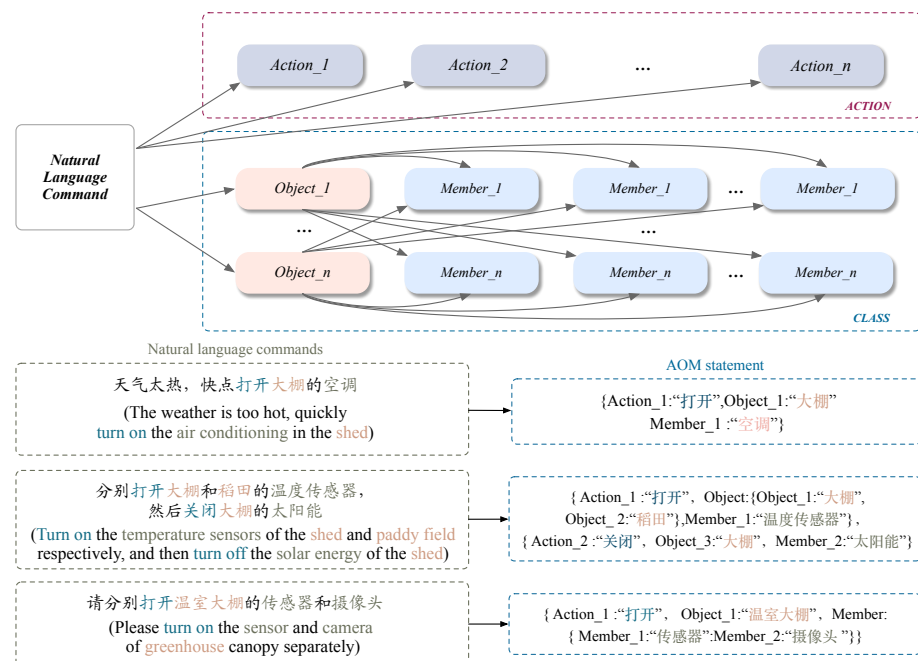


Figure 2. Sketch of a language storage framework based on dynamic tuple extraction and NL2AOM Examples.

3.2. Command Structured Parsing Algorithm and Details

This section details the implementation details of the dynamic tuple extraction-based language storage framework, where command types a and b are based on the same concept, using the TF-IDF algorithm to extract key information and the GloVe word vector pre-training model to perform the optimal similarity mapping for semantic parsing. Instruction types b, c, and d are semantically parsed using a method based on dependency syntactic analysis. The two concepts are described in detail in Sections 3.2.1 and 3.2.2, respectively.

3.2.1. Semantic Parsing Using TF-IDF Keyword Extraction Combined with GloVe Word Vectors

This section introduces the key indicator extraction process of simple natural language instructions and complex natural language instructions. In this paper, we consider simple natural language instructions as sub-instructions of complex natural language instructions, and we first cut complex natural language instructions into simple natural language instructions. Then we use the TF-IDF algorithm to extract keywords from the instructions, fill the extracted key indicators into the AOM language structure according to the lexical features, and use the word vector generated by GloVe to perform the similarity optimization of the extracted information. The basic workflow is shown in Figure 3.

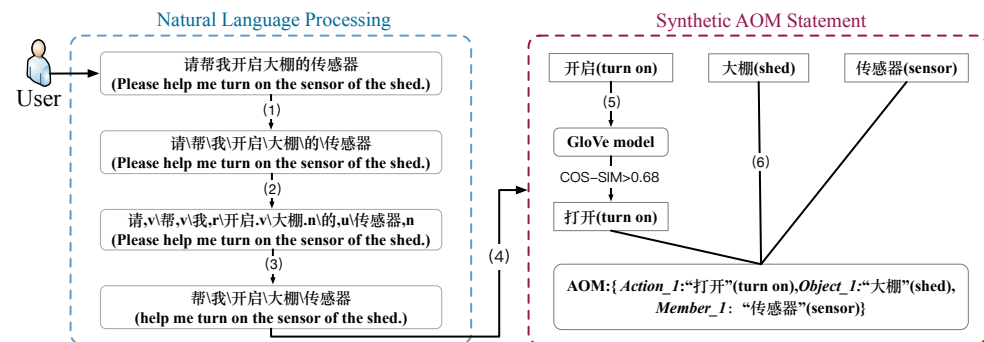


Figure 3. Workflow for the extraction of basic natural language commands.

- Step (1) represents the Chinese word separation of the input Chinese natural language commands. Unlike languages such as English, where there are spaces separating each word, there is no separation between each word in Chinese; word separation is required before subsequent processing.
- Step (2) performs lexical annotation, and in the lexical annotation result, the terms marked as /c are concatenations in the commands. Parallel conjunctions connect words, phrases, sentences, etc., such as “then”, “and”, “our”, “so”, and so on. Currently, the presence of sub-instructions is determined by considering the presence or absence of concatenation in the input natural language commands. In the proposed method, if coordinating conjunctions such as “and”, “our”, and “then” are found later, the command is considered a complex natural language command.
- Step (3) represents stop word filtering and the removal of some redundant information.
- Step (4) represents the extraction of key indicator information using the TF-IDF keyword extraction process; the details are described as follows.

First, lexical annotation is performed for natural language commands to limit the range of word attributes and improve the effectiveness of keyword extraction. In this paper, the lexical annotation is based on the lexical annotation set of BYU. A dictionary containing 98 words is defined to match the lexical properties, and the lexical annotation is completed while the words are divided. Finally, keyword extraction is completed based on lexical annotation. The main purpose is to extract the relevant attributes describing the controlled object in the command, which is achieved by stop-word elimination, word frequency-inverse text frequency algorithm, and word sense similarity calculation. Term

Frequency (TF), which indicates the frequency of the keyword, w , in the document, D_i , is calculated as:

$$TF_{w_i d_i} = \frac{\text{count}(w)}{|D_i|} \quad (1)$$

where $\text{count}(w)$ denotes the number of occurrences of keyword w , and $|D_i|$ is the number of all words in document D_i .

- Step (5) represents the optimization process of similarity calculation; for example, in this subsection, “开启” (“open”) may not be recognized, so the GloVe word vector model is used to vectorize it, calculate the cosine similarity, and find the most similar synonyms for replacement. After repeated tests, the two words with cosine similarity greater than 0.68 are synonyms. The cosine similarity of “打开” (“turn on”) and “开启” (“open”) is 0.87, so they are identified as synonyms; for the word vectors x_i and y_i , the cosine similarity is calculated as follows, where the cosine similarity takes the value interval of [0,1]. In Equation (2), the closer the cosine value of the angle between the two word vectors is to 1, the closer the semantics of these two words are.

$$\cos \langle x_i, y_i \rangle = \frac{\sum_{i=1}^n (x_i \times y_i)}{\sqrt{\sum_{i=1}^n x_i^2 \times \sum_{i=1}^n y_i^2}} \quad (2)$$

- Step (6) represents the process of synthesizing the AOM statement, which automatically fills the corresponding slot with the extracted key information, and finally converts the natural language command “请帮我开启大棚的传感器” (“Please help me turn on the sensor of the shed”) into the AOM statement {Action_1: “打开” (“turn on”), Object_1: “大棚” (“shed”), Member_1: “传感器” (“sensor”)}. In summary, we use a pseudo-algorithm to describe the above process, as shown in Algorithm 1.

Algorithm 1 Commands extraction strategy based on keywords extraction and similarity mapping.

Input: Natural language command sequences:CommandString

Output: Structured AOM Statements

```

1: Initialize AOM = ∅
2: postag = HanLP(CommandString) /*Using HanLP for lexical annotation of commands*/
3: IF postag == 'c' /*Find the conjunctive lexeme, then it is a complex natural language command*/
4:   String = split(CommandString) /*Cut complex natural language commands into basic
natural language commands*/
5:   ELSE StringList = TF-IDF(String, 'TF') /*Keyword extraction for commands */
6:   END IF
7: FOR word IN StringList
8:   IF word.postag == 'v' /*If the keyword word is a v*/
9:     Action←word
10:  ELSE IF GloVe_cossim(word, word_dict) > 0.68 /*Similarity-optimal mapping*/
11:    Action←word
12:  IF word.postag == 'ns' /*If the keyword word is a ns*/
13:    Object←word
14:  ELSE IF GloVe_cossim(word, word_dict) > 0.68 /*Similarity-optimal mapping */
15:    Object←word
16:  IF word.postag == 'nz' OR 'nc' /*If the keyword word is a nz or nc*/
17:    Member←word
18:  ELSE IF GloVe_cossim(word, word_dict) > 0.68 /*Similarity-optimal mapping */
19:    Member←word
20:  END IF
21:  AOM ← {Action_1 : “word_1”, Object: {Object_1: “word_2”, Object_2: “word_3”}, Member1_1: “word_4”} *Update AOM*/
22: END FOR
23: RETURN AOM

```

3.2.2. Structured Processing of Natural Language Commands Based on Dependency Parsing

This section describes the use of dependent syntactic analysis to generate a dependent syntactic tree from natural language instructions, combine it with a semantic parsing algorithm to complete the structured representation of natural language instructions, and automatically synthesize AOM statements. In this paper, we use the HanLP Chinese language processing tool to analyze the dependency syntax of unstructured natural language commands. The dependency syntax analysis module in HanLP uses the maximum entropy model to find the most probable dependency relationship between any two words and its probability based on the information of the words themselves, lexical properties, suffixes, and the distance between the two words, and then determines the node position of the word in the dependency tree and its relationship with the final position; the minimum spanning tree algorithm is used to obtain the entire dependency tree. We explain the common dependency relations next, as shown in Table 1.

Table 1. Common dependency.

Relations	Word Descriptions	Examples
HED	Sentence Core	Turn on the table lamp in the shed (ROOT, turn on, HED)
ATT	Definite and central words	Turning off the sensors in the shed (sensors, shed, ATT)
VOB	Object and Predicate	Please help me to turn on the nozzle of the rice field (turn on, nozzle, VOB)
COO	The same type of words	Turn on the pumps and sensors in the wheat field (pumps, sensors, COO)

Next, we introduce and analyze the dependencies of natural language commands of types c, d, and e, respectively.

Object juxtaposition natural language commands (c): For example, the semantic dependencies of the command “分别打开大棚和稻田的传感器” (“turn on the sensors of the shed and rice field respectively”) is shown in Figure 4, where the object attributes “大棚” (“shed”) and “稻田” (“rice field”) belong to the same type of member “传感器” (“sensor”) as in the command definition. “稻田” (“rice field”) are concurrently related (COO), and they point to the same type of member “传感器” (“sensor”). In this example, “打开” (“turn on”) and “传感器” (“sensor”) form a verb–object relationship (VOB), which can be populated as an action property in action based on this property. Then “大棚” (“shed”) and “传感器” (“sensor”) form a definite-in relationship (ATT), a relationship between the definite and the central word. Based on this property, the first object is determined to be “大棚” (“shed”) and is automatically populated with Object_1. Next, “稻田” (“rice field”) and “大棚” (“shed”) form a juxtaposition (COO), and the word prototype corresponding to the juxtaposition can be extracted and automatically populated into Object_2 as the second object. In this case, “打开” (“turn on”) and “传感器” (“sensor”) are in a verb–object relationship, so we can find the object “传感器” (“sensor”) based on the verb “打开” (“turn on”). The object “传感器” (“sensor”) is found and automatically populated as the first member of Member_1. Therefore, using the dependencies between languages, the structure of the synthesized AOM statement is {Action_1 : “打开”(“turn on”), Object: {Object_1: “大棚”(“shed”), Object_2: “稻田”(“rice field”)}, Member1_1: “传感器”(“sensor”)}. The algorithm describing the process of extracting this key information is shown in Algorithm 2.

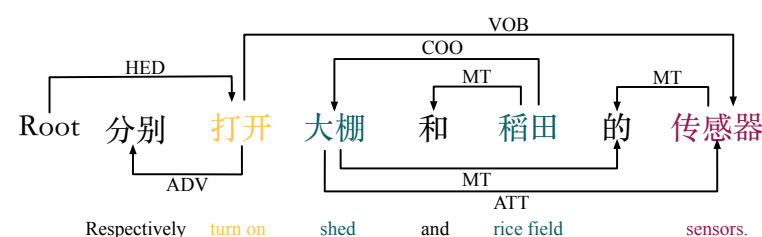


Figure 4. Example of a dependency relationship.

Algorithm 2 Semantic parsing process for the juxtaposition of natural language commands for objects.

Input: Dependency syntactic analysis tree generated by each keyword node, each node of the tree contains the original word:
word, lexical property: postag, dependency: deprel, node position dependency: head
Output: Structured AOM Statements

```

1: Initialize AOM =  $\emptyset$ 
2: FOR relation IN deprel /*Iterative dependencies*/
3:   num = num + 1 /*Automatic loop + 1 for list subscript positioning*/
4:   IF relation == 'VOB' /*Traversing the verb-object relationship*/
5:     Action ← word[head[num] − 1] /*Find the word prototypes on which the VOB relationship depends as
action attributes*/
6:   ELSE IF relation == 'COO': /*Traversing the COO relationship/
7:     Object ← word[head[num] − 1] /*Use the COO-dependent word prototype as the first object*/
8:     Object.append(word[num]) /*The COO itself corresponds to the word prototype for the second object*/
9:   ELSE IF relation == 'ATT' /*Traversing the ATT relationship*/
10:    Member ← word[head[num] − 1] /*ATT-dependent word prototypes as manipulated member attributes*/
11:  END IF
12: AOM ← {Action_1: "word_1", Object: {Object_1: "word_2", Object_2: "word_3"}, Member1_1: "word_4"} /*Update AOM*/
13: END FOR
14: RETURN AOM

```

Member-parallel natural language commands (d): For example, the dependency of command type d “分别打开大棚的传感器和摄像头”(“turn on the sensor and camera of the shed separately”) is shown in Figure 5. In this case, “传感器”(“sensor”) and “摄像头”(“camera”) are in a concurrent relationship (COO). First, we can see from the diagram that the semantic dependency between “大棚”(“shed”) and “传感器”(“sensor”) is a neutral relationship, and we determine the first object attribute “大棚”(“shed”) based on this property and automatically populate it into Object_1 as the first object. Next, the semantic dependency between the verb “打开”(“turn on”) and “传感器”(“sensor”) is a verb-object relationship (VOB), with the arrow pointing from “打开”(“turn on”) to “传感器”(“sensor”). Therefore, “传感器”(“sensor”) can be extracted as the first member through the verb-object relationship and automatically filled into Member_1. Next, we determine “摄像头”(“camera”) as the second member to be populated in Member_2 according to the concatenation relationship (COO). Finally, “打开”(“turn on”) is automatically populated as the first action attribute in Action_1 by using the same verb-object relationship. {Action_1: “打开”(“turn on”), Object_1: “大棚”(“shed”), Member:{ Member_1: “传感器”(“sensor”), Member_2: “摄像头”(“camera”)}}. The algorithm describing the process of extracting this key information is shown in Algorithm 3.

Object nested natural language commands (e): For example, the dependency of the command type e “查看大棚里土壤的温度和湿度”(“check the temperature and humidity of the soil in the shed”) is shown in Figure 6, which describes the idea of nesting objects. As we can see, “温度”(“temperature”) and “湿度”(“humidity”) are members of the object “土壤”(“soil”), and the user needs to complete the operation to check the temperature and humidity of the soil, but “土壤”(“soil”) is modified by “大棚”(“shed”), which means that the “土壤”(“soil”) is the soil of the “大棚”(“shed”). Therefore, “大棚”(“shed”) and “土壤”(“soil”) form a nested object relationship, and “大棚”(“shed”) can be identified as the first object, pointing to the second object “土壤”(“soil”). For example, the command “查看大棚里空调旁边传感器的二氧化碳的值”(“Check the CO2 value of the sensor next to the air conditioner in the shed.”) contains the user’s intention to check the carbon dioxide content, and the first object described in the middle can be identified as Object_1: “大棚”(“shed”) pointing to the second object Object_2: “空调”(“air conditioner”), and then points to the third object Object_3: “传感器”(“sensor”).

Algorithm 3 Semantic parsing process for the juxtaposition of natural language commands for members.

Input: Dependency syntactic analysis tree generated by each keyword node, each node of the tree contains the original word:
word, lexical property: postag, dependency: deprel, node position dependency: head

Output: Structured AOM Statements

```

1: Initialize AOM =  $\emptyset$ 
2: FOR relation IN deprel /*Iterative dependencies*/
3:   num = num + 1 /*Automatic loop + 1 for list subscript positioning*/
4:   IF relation == 'VOB' /*Traversing the verb-object relationship*/
5:     Action ← word[head[num] - 1] /*Find the word prototypes on which the VOB relationship depends as
action attributes*/
6:   ELSE IF relation == 'COO' /* Traversing the COO relationship /
7:     Member ← word[head[num] - 1] /*Use the COO-dependent word prototype as the first member*/
8:     Member.append(word[num]) /*The COO itself corresponds to the word prototype for
the second member*/
9:   ELSE IF relation == 'ATT' /* Traversing the ATT relationship*/
10:    Object ← word[head[num] - 1] /*Use the ATT-dependent word prototype as the object*/
11:   END IF
12: AOM ← Action_1: {"word_1", Object_1: "word_2", Member: {Member 1_1: "word_3", Member 1_2: "word_4"}}
/*Update AOM*/
13: END FOR
14: RETURN AOM

```

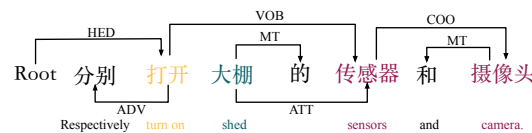


Figure 5. Example of dependency relationship.

For later analysis, we will show the result of the lexical annotation of the command “查看大棚里土壤的温度和湿度” (“Check the temperature and humidity of the soil in the shed”), as shown in Figure 6. Therefore, we can conclude that if we find a keyword with the lexicality /f, the command is considered an object-nested query. From the dependency diagram, we can see that the keyword “里” (“inside”) before “大棚” (“shed”) in the f-terminology forms a neutral relationship (ATT), while “土壤” (“soil”) points to “里” (“inside”). Therefore, the nested object can be found by locating the keyword with the f-word nature. In the definition given in this paper, the keyword dependent on the f lexicality is regarded as the initial object in the nesting process, and the keyword dependent on it is regarded as the object pointed to in the nesting process, according to which Object_1: “大棚” (“shed”), Object_2: “土壤” (“soil”). The keywords “查看” (“check”) and “温度” (“temperature”) form a verb-object relationship (VOB), and we can determine the member “温度” (“temperature”) according to the (VOB), and fill it into Member_1. In addition, “温度” (“temperature”) and “湿度” (“humidity”) form a concurrent relationship (COO), which is extracted as the second object and automatically populated in Member_2. Finally, according to the verb-object relationship, the verb “查看” (“check”) is extracted as the first action and automatically filled into Action_1. The final synthesized AOM statement is {Action_1: “查看” (“check”), Object_1: “大棚” (“shed”), Object_2: “土壤” (“soil”), Member: {Member_1: “温度” (“temperature”): Member_2: “湿度” (“humidity”)}}. The algorithm describing the key process of parsing is shown in Algorithm 4.

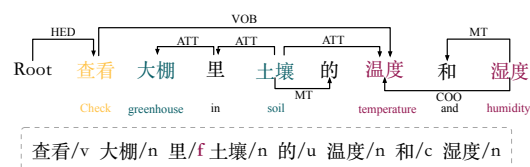


Figure 6. Example of dependency relationship and natural language command lexical annotation results.

Algorithm 4 Semantic parsing process for Object nested natural language commands.

Input: Dependency syntactic analysis tree generated by each keyword node, each node of the tree contains the original word: word, lexical property: postag, dependency: deprel, node position dependency: headOutput: Structured AOM Statements

```

1: Initialize AOM =  $\emptyset$ 
2: FOR relation IN deprel /*Iterative dependencies*/
3:   num = num + 1 /*Automatic loop + 1 for list subscript positioning*/
4:   IF relation == 'VOB' /*Traversing the verb-object relationship*/
5:     Action ← word[head[num] - 1] /*Find the word prototypes on which the
VOB relationship depends as action attributes*/
6:   ELSE IF relation == 'ATT'
7:     NestingObject ← word[num] /*First traversal of ATT dependencies
as nested object*/
8:   ELSE IF (postag[num] == 'f') AND (relation == 'ATT') /*Find the prototype
of the word lexical category for f*/
9:     NestedObject ← word[head[num] - 1] /*Dependent word prototype of
word lexical f is the nested object*/
10:    Member ← word[head[head[num] - 1]] /*Find the manipulated member by the nested object*/
11:  END IF
12:  AOM ← {Action_1: "word_1", Object_1: "word_2", Member: {Member 1_1: "word_3", Member 1_2: "word_4"}}
/*Update AOM*/
13: END FOR
14: RETURN AOM

```

4. Natural Language Interfaces

Neural networks have a wide range of applications in the field of nonlinear models, and the Seq2Seq model is particularly suitable for nonlinear neural systems because it can naturally handle variable-length input and output sequences [26]. To enhance the scientific generalization of the model, we use a dynamic tuple extraction framework to generate a natural language command-AOM dataset to train the deep learning model. We take raw natural language commands, e.g., “天气炎热，请先帮我打开大棚的空调，接着帮我查看一下现在大棚的温度是多少” (“It is hot, please turn on the air conditioner of the shed for me first, and then check the temperature of the shed for me”) as input, and output the formal command “打开大棚空调查看大棚温度” (“Turn on the air conditioner of the shed and check the temperature of the shed”), and finally, the formal command rules are synthesized into AOM statements. This allows us to learn the rules and realize the concept of sequence-to-sequence.

4.1. Seq2seq Model

For the input sequence $x = \{x_1, x_2, \dots, x_t\}$, the model estimates the conditional probability distribution $p(y|x)$ for all possible output sequences $x = \{x_1, x_2, \dots, x_{t'}\}$, where the sequence lengths t and t' are allowed to possess differences between them. In task NL2AOM of this paper, x is an input natural language command, and y is a structured AOM statement, which can also be a sequence command. We will convert it to a target output sequence using a specific specification command.

The encoder part is implemented by a generic recurrent neural network (RNN). In this paper, we use recurrent neural networks [27] and their variants, such as LSTM [28], GRU [29], BiLSTM [30], and BiGRU [31], to encode the input sequence x into a vector of fixed dimensions (the character representation in the equations below continues to use RNN).

$$h_0 = \text{RNN}(x) \quad (3)$$

The output of each hidden layer is obtained, and then a semantic vector is generated.

$$C = q(h_1, h_2, h_3, \dots, h_t) \quad (4)$$

In the decoder stage, we have to predict the next word based on the given semantic vector and output sequence $y_t = (y_1, y_2, y_3, \dots, y_t)$, i.e.,:

$$y_t = \operatorname{argmax} P(y_t) = \prod_{t=1}^T p(y_t | \{y_1, y_2, y_3, \dots, y_{t-1}\}) \quad (5)$$

Detailed model training details are given in the experimental subsection.

4.2. Seq2seq Paraphrase Model with BERT

In this section, we construct a natural language interface for the HMI approach in agriculture and build an adapted deep learning model to solve the NL2AOM problem; the model architecture is shown in Figure 7. A BERT [32]-based Seq2Seq [33] model controls the context of the prediction conditions by using a shared Transformer [34] network and by utilizing specific self-attention masks. We next describe the self-attention mechanism [35] in detail in Section 4.2.1, and Section 4.2.2 describes the principle of the Seq2Seq model built based on BERT.

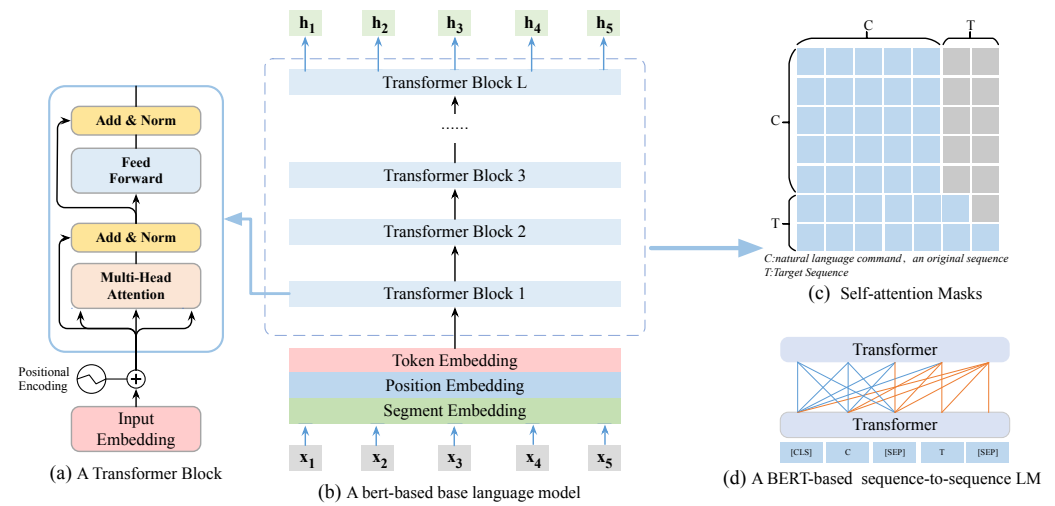


Figure 7. BERT-based Seq2Seq language model.

4.2.1. Self-Attention Network

The attention mechanism is widely used in various tasks of natural language processing based on deep learning, and the self-attention mechanism has become a hot research topic of neural network attention. The calculation process of **attention** and **multi-head-attention** are shown in Figure 8a,b. The calculation formula of attention is shown below.

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (6)$$

where $Q \in R^{n \times d_k}$, $K \in R^{m \times d_k}$, $V \in R^{m \times d_v}$. Without considering the softmax layer, we can consider the attention layer as encoding a sequence Q of $n \times d_k$ into a new sequence of $n \times d_v$. Let us make the metric more detailed, Q , K , and V are shorthand for query, key, and value, respectively, where K and V is a one-to-one correspondence, and they are key–value relationships. The above equation calculates the similarity between Q and each V by calculating the inner product of Q and each K and performing softmax, and then weighting and summing to get a d_v -dimensional vector, where the factor $\sqrt{d_k}$ plays a regulating role of not making the inner product too large.

Regarding the multi-head attention mechanism, which is a refinement of the attention mechanism, the approach is to map Q , K , and V through the parameter matrix, and then in performing the attention calculation, the process is repeated h times, and finally, the calculation results are stitched together. The formula is as follows:

$$head_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (7)$$

Here $WQ_i \in Rd_k \times d^k$, $WK_i \in Rd_k \times d^k$, $WV_i \in Rd_k \times d^k$ and then we have :

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_h) \quad (8)$$

Finally, a sequence $n \times hd_v$ is obtained.

Self-attention is a kind of *attention*, as $\text{attention}(X, X, X)$, X is also the input sequence; that is, the attention is calculated inside the sequence to find the connection inside the sequence. Its calculation formula is as follows:

$$Y = \text{MultiHead}(X, X, X) \quad (9)$$

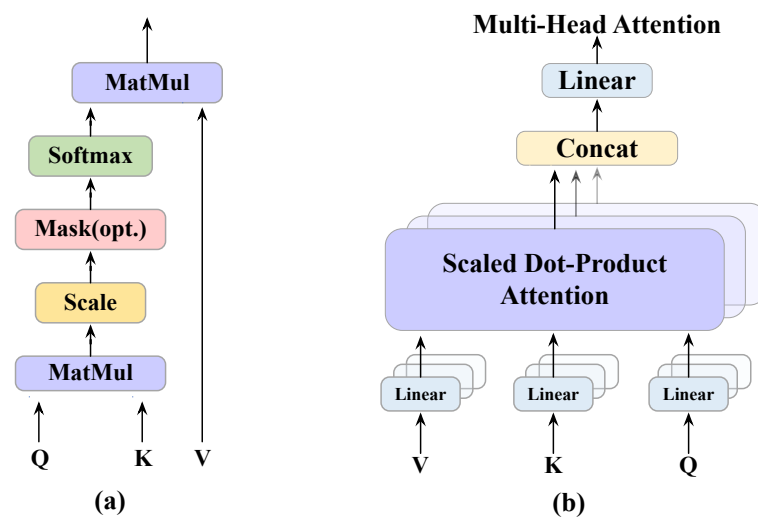


Figure 8. (a) The attention mechanism. (b) The abstract computational flow of the multi-headed attention mechanism.

In summary, it can be seen that the attention mechanism, compared to recurrent neural networks (RNN), requires global information to be obtained by step-by-step recursion and takes the direct way of computing global information. *PositionEmbedding*, which numbers each position and then each number corresponds to a vector, introduces certain position information to each word by combining the position vector and the word vector; in this way, the attention mechanism can distinguish words in different positions. The formula for calculating the position vector is shown below.

$$PE_{(pos, 2i)} = \sin\left(pos/10,000^{2i/d_{\text{model}}}\right) \quad (10)$$

$$PE_{(pos, 2i+1)} = \cos\left(pos/10,000^{2i/d_{\text{model}}}\right) \quad (11)$$

The location-embedding matrix is shown in Figure 9, and to clearly describe this variation, we define the maximum sentence length as 50 and the vector dimension as 50 dimensions and find that this period variation will become slower and slower as the vector dimension increases while generating texture-containing location information with the period of the location function from 2π to $10,000 \times 2\pi$ and each location in the embedding dimension becomes a combination of sin and cos function values with different periods. Thus generating unique texture location information allows the model to learn the dependencies between locations and the temporal properties of natural language.

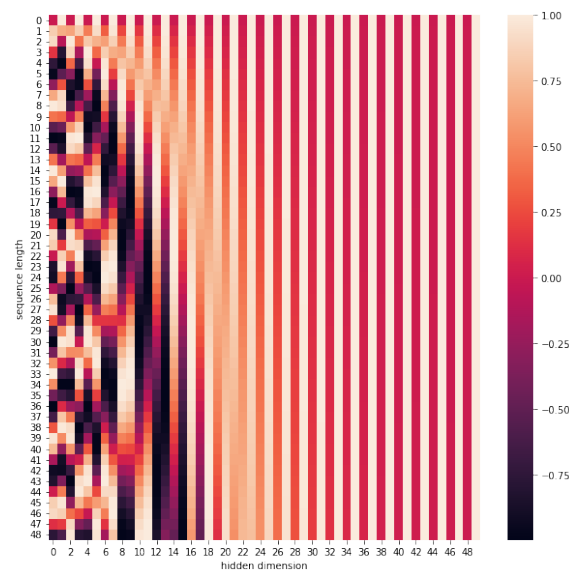


Figure 9. Position-embedding matrix.

4.2.2. Model Details

The architecture of the proposed BERT-based Seq2Seq language model is shown in Figure 7. From Section 4.2.1, we know that the self-attention mechanism is Q , K , and V , and they are all the same vector sequence after linear changes, while Transformer is the combination of self-attention and Position-Wise fully connected layers, and Transformer is based on the attention vector sequence to vector sequence change. Therefore, this subsection analyzes the NL2AOM task in terms of the language model, the mask approach of the attention matrix, and how to use the BERT pre-trained model to build a natural language interface with scientific generalization capability and extensibility.

Language Model: The language model is an unconditional generative model, where the one-way language model is equivalent to taking the training corpus and computing it using conditional probabilities.

$$p(x_1, x_2, x_3, \dots, x_n) = p(x_1)p(x_2 | x_1)p(x_3 | x_1, x_2) \dots p(x_n | x_1, \dots, x_{n-1}) \quad (12)$$

RNNs are well suited for the sequential nature of natural language due to their unique recursive form. However, the attention mechanism and its variants can still achieve sequential prediction if the appropriate mask is chosen. As shown in Figure 10, each row of the attention matrix represents the output, and each column represents the input, where the blue squares represent the input and the gray squares represent the output. Output x_1 is thus only related to the starting token $\langle s \rangle$, while output x_2 in the second row is related to $\langle s \rangle$ and x_1 , and so on, to achieve the idea of recursion.

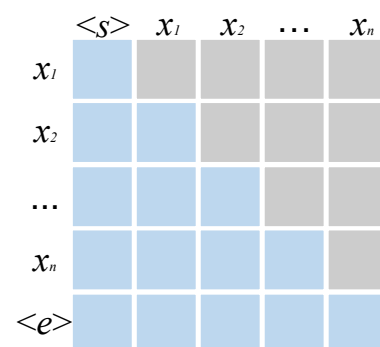


Figure 10. Attention Matrix.

For the requirements of this paper, we consider the natural language command “帮我打开大棚的灯” (“help me turn on the shed light”) as input and the target command “打开大棚灯” (“turn on the shed light”), and splice the two sentences as (CLS) “帮我打开大棚的灯” (“help me turn on the shed lights”) (SEP) “打开大棚灯” (“turn on the shed light”) (SEP). From (CLS) “帮我打开大棚的灯” (“help me turn on the shed light”) (SEP) to predict “打开大棚灯” (“turn on the shed light”) word by word until (SEP) appears directly. The structure of the attention matrix is shown in Figure 11. However, the input part of the Seq2Seq sequence model is bidirectional, and the output part is unidirectional; therefore, in contrast to Figure 11a, which adds “帮我打开大棚的灯” (“Help me turn on the lights in the shed”) to the prediction range, Figure 11b chooses to process this part directly in mask. Therefore, the attention of the input port is bidirectional, and the attention of the output part is unidirectional, satisfying the Seq2Seq requirement, which is the self-attention mask process represented in Figure 7c.

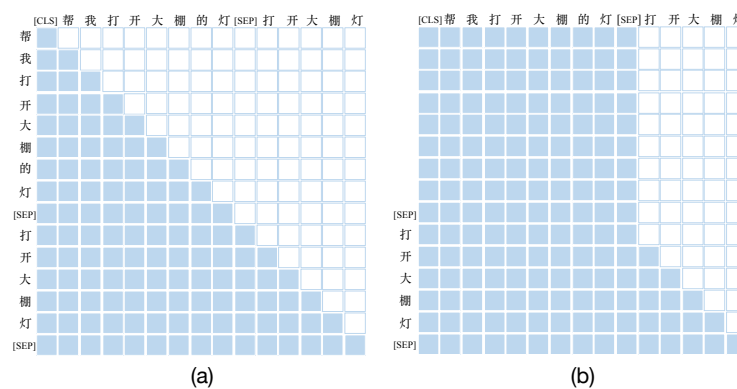


Figure 11. (a) Attention mechanism matrix for lower delta masks. (b) Attention mechanism matrix for special masking mechanism

The input of the above model is a piece of sequential text, which exists in pair form, with the original natural language command and the target generating the natural language command. A (CLS) token is added to the beginning of the input, and a (SEP) token is added to the end. The input representation is shown in Figure 7b and includes segment embedding, position embedding, and token embedding. The network layer consists of 24 Transformer networks, and the input vector $\{x_1, x_2, \dots, x_i\}$ is first encoded as $Hidden = \{x_1, x_2, \dots, x_{|x|}\}$, which is then encoded by the Transformer network, where the encoded output of each layer is :

$$Hidden^l = \text{Transformer} \left(Hidden^{l-1} \right) \quad (13)$$

Then, the self-attention mask idea is used to control the attention range of the predicted words, where the approach of this paper constructs a mask matrix M . In this matrix, the idea is to perform the inner product calculation after the matrix summation of the mask matrix M , Q , and K , where the value of the mask in M needs to be negative infinity, and the value after softmax is 0, indicating it is of no concern; it will then be masked and processed. The part that needs attention is 0, which does not affect the size of the original matrix. For the i -th Transformer network layer, the output of the self-attentive mechanism is :

$$Q = \text{Hidden}^{i-1} W_i^Q, K = \text{Hidden}^{i-1} W_i^K, V = \text{Hidden}^{i-1} W_i^V \quad (14)$$

$$\text{Self-Attention}_1 = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} + M \right) V_i \quad (15)$$

The specific model training effects are shown in the experimental section.

5. Experiment

The experimental part is carried out in the following sections. (a): First, the data collection effect of the dynamic tuple extraction framework is analyzed for its accuracy, precision, recall value, and F-Score value. The quality of data collection determines how well the subsequent Seq2Seq method fits. (b): The performance of the traditional Seq2Seq-based model and the BERT-based Seq2Seq model is analyzed for the effectiveness of data fitting, still in terms of accuracy, precision, recall value, and F-score. (c): It is tested in the real agricultural HMI domain to analyze the effect of command semantic recognition and the response time of HMI. The evaluation metrics of the model performance are given in Section 5.1; the performance of the dynamic tuple extraction framework is evaluated in Section 5.2; the fitting effect of the Seq2Seq model on the data is analyzed in Section 5.3. An example in the field of agricultural HMI is given and analyzed in Section 5.4.

5.1. Model Performance Evaluation Metrics

The NL2AOM performance is evaluated in terms of accuracy, precision, recall value, and F-value, which are obtained by manual determination in this paper. The definitions of each formula are as follows.

Accuracy: indicates the proportion of all parsing processes that correctly parse natural language commands into AOM statements. For example, “打开大棚的吊灯” (“turn on the chandelier of the shed”) is converted to an AOM statement as “{Action_1: “打开” (“turn on”), Object_1: “大棚” (“shed”), Member_1: “吊灯” (“chandelier”)}, is considered to be the correct analysis, other forms are wrong. The formula is shown where C indicates the number of correctly parsed commands and A indicates the number of all parsed commands.

$$\text{Accuracy} = \frac{C}{A} \quad (16)$$

Precision: indicates the proportion of the correct key information in the generated AOM statement to the actual generated information, consistent with the above example, if the generated AOM statement is {Action_1: “open”, Object_1: “shed”, Member_1: “chandelier”}, then the correct indicator generated is Action_1: “open”, Object_1: “shed”, while Member_1: “Chandelier” should be “shed”, then the accuracy rate is calculated as 2/3. The formula is shown where P denotes the number of all key indicators in the parsed AOM statement, and S denotes the number of successfully parsed indicators in the AOM statement.

$$\text{Precision} = \frac{S}{P} \quad (17)$$

Recall: is the proportion of the correct key information in the generated AOM statement to the actual key information contained in the original command; for example, if the original natural language is “先打开大棚的空调，再查看大棚的温度是多少” (“first turn on the air conditioner of the shed, and then check what the temperature of the shed is”), the theoretical generated AOM statement should be [{Action_1: “打开” (“turn on”), Object_1: “大棚” (“shed”), Member_1: “空调” (“air conditioner”)}, {Action_1: “查看” (“check”), Object_1: “大棚” (“shed”), Member_1: “温度” (“temperature”)}], if only {Action_1: “打开” (“turn on”), Object_1: “大棚” (“shed”), Member_1: “空调” (“air conditioner”)}; and the recall value is calculated as 3/6. The calculation formula is shown, where R denotes the original actual number of key messages contained in the natural language command.

$$\text{Recall} = \frac{S}{R} \quad (18)$$

F – score: this evaluation index responds to the reconciled average of accuracy and recall, which can reflect the good or bad performance of NL2AOM in a comprehensive way, calculated as follows.

$$F - score = \frac{2 \times precision \times recall}{precision + recall} \quad (19)$$

5.2. Experimental Performance Evaluation of Language Frameworks Based on Dynamic Tuple Extraction

We construct our test dataset through a low-cost crowdsourcing approach. For experimental flexibility, we use five natural language command counterparts for one AOM statement and crowdsource a total of 1000 natural language command-AOM statement pairs, including 200 natural language commands of command types a, b, c, d, and e, respectively. Some of the samples of the test dataset are shown in Table 2.

Table 2. Partial example of NL2AOM crowdsourced dataset.

No.	Type	Natural Language Commands	AOM Statements
1	a	请帮我打开大棚的空调 (Please help me turn on the air conditioning in the shed)	{Action_1: “打开” (“turn on”), Object_1: “大棚” (“shed”), Member_1: “空调” (“air conditioning”)}
2	b	先打开大棚的空调，再查看大棚的温度是多少 (First turn on the air conditioning of the shed, and then check what the temperature of the shed is)	{ Action_1: “打开” (“turn on”), Object_1: “大棚” (“shed”), Member_1: “空调” (“air conditioning”)}, {Action_1: “查看” (“check”), Object_1: “大棚” (“shed”), Member_1: “温度” (“temperature”)}
3	c	请帮我同时打开稻田和麦田的水泵 (Please help me turn on the water pump in both the rice and wheat fields)	{ Action_1: “打开” (“turn on”), Object: {Object_1: “稻田” (“rice fields”), Object_2: “麦田” (“wheat fields”)}, Member_1: “水泵” (“pump”)}
4	d	分别打开稻田的喷头和传感器 (Turn on the nozzle and sensor of the rice field separately)	{Action_1: “打开” (“turn on”), Object_1: “稻田” (“rice fiel”), Member: {Member_1: “喷头” (“nozzle”), Member_2: “传感器” (“sensor”)}}
5	e	请帮我查看稻田里土壤的温度和湿度 (Please help me to check the temperature and humidity of the soil in the rice field)	{Action_1: “查看” (“check”), Object_1: “稻田” (“rice fiel”), Object_2: “土壤” (“soil”), Member: {Member_1: “温度” (“temperature”), Member_2: “湿度” (“humidity”)}}
...	...	...	...

To explore the performance of the model on different instructions, various numbers of samples (50/100/150/200) are randomly selected for testing and are input to the corresponding semantic parsing algorithm, and the generated AOM statements are compared with the actual AOM statements to calculate the relevant performance evaluation index values and the maximum time spent in the instruction parsing process. The test results are shown in Tables 3–7. The trends of their four performance evaluation indicators are shown in Figures 12a–d, respectively.

Table 3. Command type a Experimental results.

No.	Number	Accuracy	Precision	Recall	F-Score	Time Taken
1	50	96.21%	95.52%	94.37%	94.94%	0.92 s
2	100	95.57%	93.39%	92.57%	92.98%	0.93 s
3	150	94.29%	93.57%	92.89%	93.23%	0.95 s
4	200	93.87%	92.24%	91.19%	91.71%	0.94 s

Table 4. Command type b Experimental results.

No.	Number	Accuracy	Precision	Recall	F-Score	Time Taken
1	50	96.22%	95.37%	94.59%	94.98%	0.97 s
2	100	95.29%	93.56%	92.29%	92.92%	0.98 s
3	150	93.21%	92.29%	92.27%	92.28%	0.97 s
4	200	92.27%	91.48%	90.57%	91.02%	0.95 s

Table 5. Command type c Experimental results.

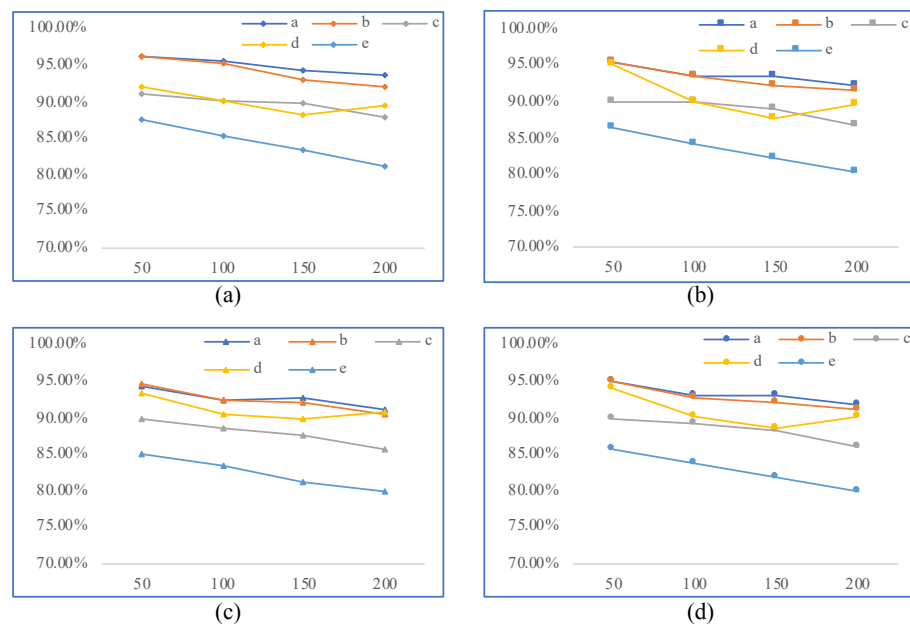
No.	Number	Accuracy	Precision	Recall	F-Score	Time Taken
1	50	91.15%	90.07%	89.87%	89.96%	1.97 s
2	100	90.27%	89.97%	88.53%	89.24%	1.98 s
3	150	89.87%	88.87%	87.78%	88.32%	1.94 s
4	200	87.79%	86.64%	85.57%	86.10%	1.98 s

Table 6. Command type d Experimental results.

No.	Number	Accuracy	Precision	Recall	F-Score	Time Taken
1	50	92.16%	95.10%	93.27%	94.18%	1.86 s
2	100	90.10%	89.81%	90.65%	90.22 %	1.96 s
3	150	88.15%	87.58%	89.91%	88.73%	1.96 s
4	200	89.47%	89.58%	90.72%	90.15%	1.87 s

Table 7. Command type e Experimental results.

No.	Number	Accuracy	Precision	Recall	F-Score	Time Taken
1	50	87.57%	86.52%	85.12%	85.81%	2.02 s
2	100	85.47%	84.12%	83.39%	83.75%	2.06 s
3	150	83.39%	82.21%	81.19%	81.70%	2.06 s
4	200	81.27%	80.23%	79.81%	80.02%	2.07 s

**Figure 12.** (a) Accuracy result graph of the commands. (b) Precision result graph of the commands. (c) The resulting graph of the recall value of the commands. (d) F-score result graph of the commands.

The following points can be obtained from the experimental result charts.

- With the increase in the number of input commands, the performance indicators of the generated AOM intermediate language are decreased, where the parsing performance of command types a and b are above 90%, and the parsing time is between 0.9 and 1 s. It can be considered that the TF-IDF algorithm combined with the use of the Glove word vector to calculate the cosine similarity has good results in the parsing of these two types of commands. The average response time of instruction type b is slightly longer than that of instruction type a. The main reason is the time consumption

of the process of cutting complex natural language commands into basic natural language commands. The good parsing performance is mainly because the commands themselves generate simple AOM statements in the form of a basic triplet. Therefore, this method is reliable for generating NL2AOM datasets for commands a and b.

- Command types c and d have index values above 85% when generating the AOM intermediate language, and the change curves of their F-score values overlap significantly when the number of input commands is 100 to 150, indicating that Algorithms 2 and 3 are more similar in performance, mainly the characteristics of these two commands are more similar; the former belongs to object juxtaposition, and the latter belongs to member juxtaposition. The response time of generating AOM intermediate language is between 1.85 and 2.0 s, so the training data can be collected effectively.
- Command type e has index values for the generation of close to 80% AOM intermediate language; the main reason for consideration is the complexity of object nesting. For example, “查看大棚里传感器的温度” (“check the temperature of the sensor in the shed”) where “大棚” (“shed”) and “传感器” (“sensor”) are objects, and “请帮我打开大棚的传感器” (“please help me open the shed’s sensor”) is a member of the “传感器” (“sensor”), which is a common cause of ambiguity. Therefore, its performance is the lowest among the five types of commands. The parsing response time is more than 2 s. Therefore, when collecting the NL2AOM dataset of type e commands, we need to manually screen out the non-conforming pairs and normalize them to provide high-quality datasets for the subsequent deep learning training.
- Combining the above experimental results, the data collection method proposed in this paper can collect high-quality datasets in a low-cost and efficient manner. It provides the basis for the subsequent training of deep learning models.

5.3. Evaluation of Sequence-to-Sequence Based Semantic Parsing Models

Part of the data in this domain is obtained from CHINESEQCI-TS DATASET [24], which contains 600 natural language commands in the agricultural domains, combined with the 1000 natural language command-AOM data pairs crowdsourced in Section 5.2. Considering the good performance of the dynamic tuple extraction framework, volunteers were sought to input common natural language commands in agriculture, and 100 natural language command-AOM statement pairs for each of command types a, b, c, d, and e were automatically generated. The results were corrected to the target canonical form. A total of 2100 natural language command-AOM statement pairs are included; each natural language command corresponds to a unique AOM statement. We divided the training and test sets in the ratio of 8:2.

In this paper, our approach is compared with other Seq2Seq models for experiments. Section 5.3.1 evaluates the accuracy of Action, Object, and Member segments generated on the training and test sets. Section 5.3.2 compares the overall synthetic AOM utterance accuracy, precision, recall, and F-score values.

5.3.1. Segment Generation Performance Evaluation

The experimental results are shown in Table 8.

The following points can be obtained from the above table.

- In the experimental tables generated by the fields, the accuracy of the method proposed in this paper is a lot higher than the other four methods, with the training set accuracy reaching 86.43%, 61.13%, and 86.43% in the three fields, respectively. The test set accuracies reached 85.24%, 60.24%, and 85.24%, respectively.
- The main reason for the leading accuracy of field extraction compared with other models is that the self-attention mechanism inside BERT combines with the mask mechanism, which can realize the recursive concept based on the perception of global information, and since the training instructions are of statement-level size, the self-attention mechanism is more capable of capturing the connections inside the sequence. Thus the field extraction effect is ahead of the classical sequence model.

- The reason for the low accuracy of the extracted field Object is that the indicator can easily cause ambiguity; that is, the same keyword can have different indicators in different instructions; for example, “查看大棚的空调温度是多少” (“check what is the temperature of the air conditioner in the shed”). The “空调” (“air conditioner”) index of this command belongs to Object_2, which forms a nested relationship with the “大棚” (“shed”). However, “请帮我打开一下大棚的空调” (“Please help me turn on the air conditioner in the shed”), where “空调” (air conditioner”) belongs to Member_1 and forms a verb–object relationship (VOB) with “打开” (“turn on”). Therefore, the accuracy rate is relatively low.

Table 8. Segments Generation Accuracy.

Fields	Model Name	Accuracy(Train)	Accuracy(Test)
Action	Seq2Seq(LSTM)	45.12%(758/1680)	42.62%(179/420)
	Seq2Seq(GRU)	41.43%(696/1680)	39.52%(166/420)
	Seq2Seq(BiLSTM)	48.87%(821/1680)	45.48%(191/420)
	Seq2Seq(BiGRU)	52.02%(874/1680)	44.52%(187/420)
	Seq2Seq(BERT)(Our method)	86.43%(1452/1680)	85.24%(358/420)
Object	Seq2Seq(LSTM)	27.20%(457/1680)	26.90%(113/420)
	Seq2Seq(GRU)	31.01%(521/1680)	29.05%(122/420)
	Seq2Seq(BiLSTM)	36.37%(611/1680)	34.76%(146/420)
	Seq2Seq(BiGRU)	35.77%(601/1680)	33.57%(141/420)
	Seq2Seq(BERT)(Our method)	61.13%(1027/1680)	60.24%(253/420)
Member	Seq2Seq(LSTM)	45.12%(758/1680)	42.62%(179/420)
	Seq2Seq(GRU)	41.43%(696/1680)	39.52%(166/420)
	Seq2Seq(BiLSTM)	48.87%(821/1680)	45.48%(191/420)
	Seq2Seq(BiGRU)	52.02%(874/1680)	44.52%(187/420)
	Seq2Seq(BERT)(Our method)	86.43%(1452/1680)	85.24%(358/420)

5.3.2. Overall Synthetic AOM Performance Evaluation

The experimental results are shown in Table 9.

Table 9. Experimental comparison results of the language models.

No.	Model Name	Accuracy	Precision	Recall	F-Score
1	Seq2Seq(LSTM)	32.17%	38.19%	37.21%	37.69%
2	Seq2Seq(GRU)	33.38%	34.57%	33.27%	33.91%
3	Seq2Seq(BiLSTM)	41.27%	47.89%	43.57%	45.53%
4	Seq2Seq(BiGRU)	40.83%	47.24%	46.87%	47.05%
5	Seq2Seq(BERT)(Our method)	78.71%	79.21%	78.98%	79.09%

The following points can be obtained from the above table.

- In the table of synthetic AOM performance evaluation values, the method Seq2Seq with BERT in this paper achieves 78.71% accuracy, 79.21% precision, 78.98% recall, and 79.09% F-score values on the NL2AOM task. Its accuracy gains are 46.54%, 45.33%, 37.44%, and 37.88% for Seq2Seq(LSTM), Seq2Seq(GRU), Seq2Seq(BiLSTM), and Seq2Seq(BiGRU), respectively. The main reason is that LSTM, BiLSTM, GRU, and BiGRU are all variants of RNN, which can solve the recursive timing problem well but cannot sense the information globally. The recursive timing feature is implemented. This results in a higher accuracy rate on the sequence-to-sequence task from natural language commands to AOM statements.
- The experimental results also show that the dynamic tuple extraction-based language framework proposed in this paper can collect high-quality training datasets efficiently,

quickly, and at a low cost. Further, the proposed BERT-based Seq2Seq model, based on the use of self-attention masks, can both globally acquire natural language instruction information and implement the sequence-to-sequence concept, thus enabling the better implementation of the NL2AOM task and providing a more robust baseline.

5.4. Agricultural Human–Machine Interaction Interface

In this section, we wrote the agricultural HMI, as well as the application for the PC platform and the mobile platform. The interaction algorithm is implemented by relying on the Rapid SCADA cloud platform, and the transfer of parameters is performed using socket communication. Three scenarios are defined in the interface, i.e., greenhouse, cornfield, and paddy field, where five devices are set up in the greenhouse, i.e., ventilation fans, lights, cameras, and sensors; lights, cameras, sprinklers and sensors are set up in the cornfield; and sprinklers, cameras, lights and sensors are set up in the paddy field.

On the desktop platform, there is an indicator light next to the device; the color is gray for off status and green for on status. On the SCADA platform server, the corresponding status of the device will be displayed on the interface. We test two commands here, as shown in Figure 13. Command 1 is “请帮我打开大棚和稻田的吊灯” (“Please turn on the chandeliers of the greenhouse and paddy field for me”). You can see on the screen that the indicator lights of the chandeliers in the shed and paddy field are changed to green. Command 2 is “请帮我关闭稻田的摄像头，然后打开玉米地的水泵，最后打开稻田的传感器” (“Please help me turn off the camera in the paddy field, then turn on the sprinklers in the corn field, and finally turn on the sensor in the paddy field”). On the screen, you can see that the indicator light for the camera in the rice field is gray, the indicator light for the sprinkler in the corn field is green, and the indicator light for the sensor in the rice field is green, next to which the soil moisture is displayed.

We also tested it on the mobile platform, as shown in Figure 14. First, we enter command 1 on the HMI page: “请帮我打开大棚里的摄像头和传感器，再关闭玉米地里的摄像头” (“Please help me to turn on the camera and sensor in the greenhouse and then turn off the camera in the corn field”). As you can see on the Device Control page, the cameras and sensors in the shed are turned on, the relative humidity of the soil is displayed next to the humidity sensor, and the cameras in the cornfield are turned off. Then, we enter command 2: “打开大棚里的风扇，再关闭玉米地的灯” (“Turn on the fans in the greenhouse and then turn off the lights in the corn field”). We can see on the device control screen that the switch for the fan in the shed is turned on, and the switch for the light in the cornfield is turned off.

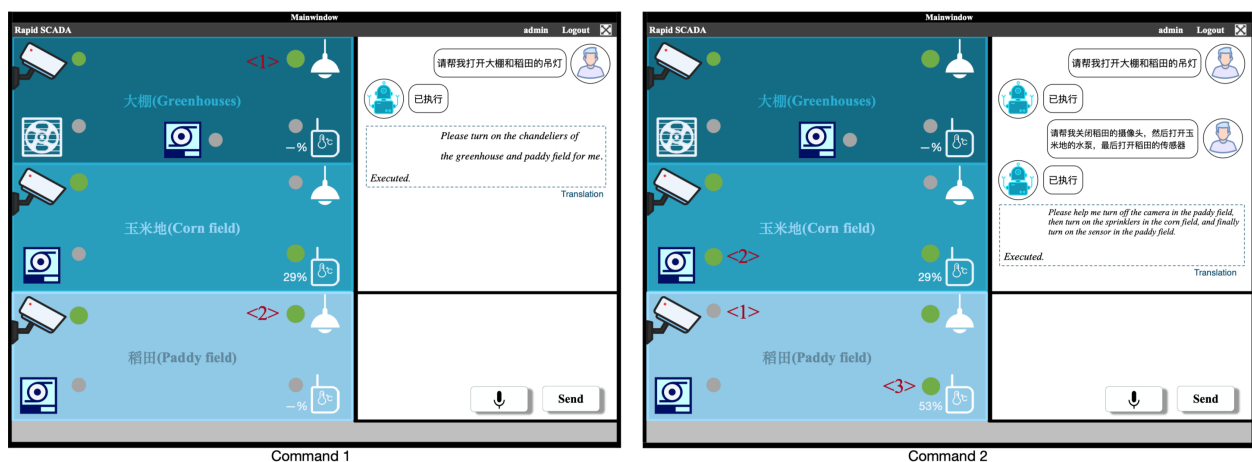


Figure 13. Desktop platform human–machine interaction interface in agriculture.

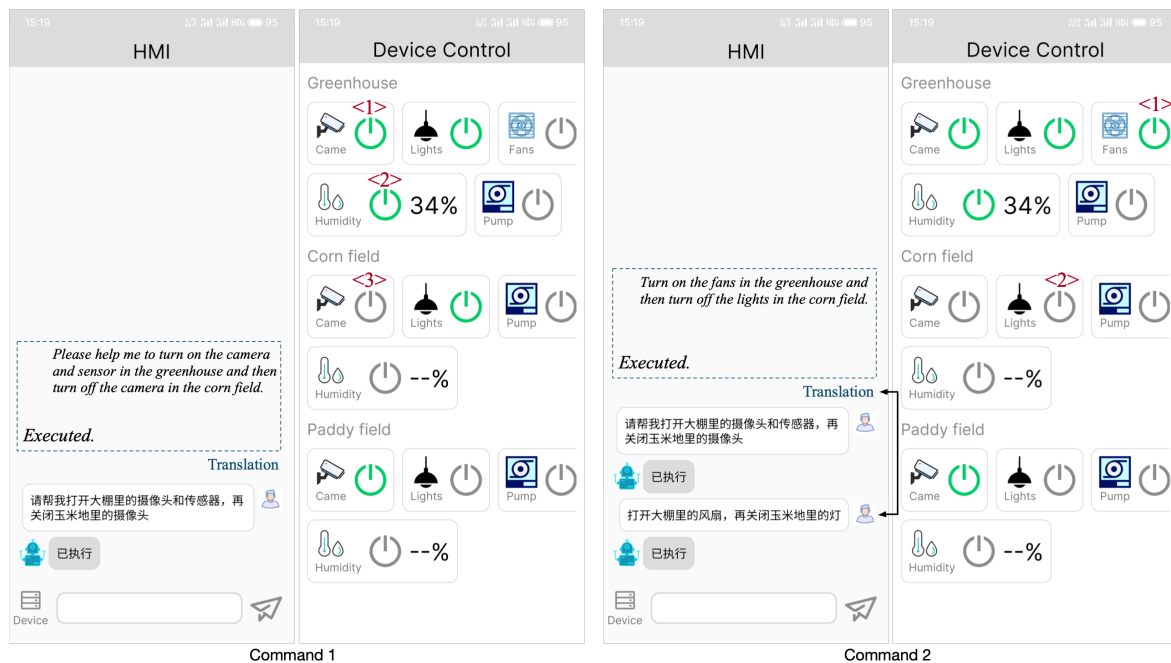


Figure 14. Mobile platform human-machine interaction interface in agriculture.

In summary, relying on the SCADA cloud platform, we can apply our model to agricultural measurement and control systems. Through our tests, the time from the command received from the SCADA cloud platform to the start of the measurement and control device is between 0.42 and 0.71 s, and the average time for the model to parse the voice command is 2.61 s. Therefore, the time from the user sending the voice command to the response of the measurement and control device is about 3.03 to 3.32 s. This response time is appropriate in modern human-machine interaction. This proves the practicability of our whole model.

6. Conclusions

In this paper, we propose a human-machine user interface for the agricultural domain, which first synthesizes the natural language commands entered by the user into a structured AOM statement (Action-Object-Member) through a new approach based on a dynamic tuple language framework to collect training data. Constructing NL2AOM, i.e., NL command-AOM call pairs, for the HMI approach interface, the method achieves an overall average accuracy, precision, recall, and F-score value of 85.21%, 87.65%, 86.67%, and 87.23% for command parsing, respectively. Then, we propose an end-to-end framework to improve the Seq2Seq model based on BERT, which uses a special mask mechanism to implement the sequence-to-sequence concept based on self-attention to obtain global information about the commands. It enables NL2AOM to learn the semantic mapping from fuzzy and unstructured NL commands to formal and structured intermediate language AOM calls. Thus, a natural user interface with high scientific generalization capability and scalability is constructed. The experimental results show that the values of each evaluation metric when the method was tested were 78.71%, 79.21%, 78.98%, and 79.09%. The experimental surface shows that the method of this paper has good performance indexes in agricultural human-machine interaction, and the response time of human-machine interaction is relatively rapid.

In this work, we are using the Chinese natural language commands dataset from an agricultural measurement and control environment; this leads to the limitation of our model to agricultural measurement and control environment for Chinese natural language commands. In the future, we will consider migrating our models to different domains. Examples are the smart home domain and the industrial domain, and we can try other languages, such as English and Spanish. We consider a knowledge graph perspective

to store data and make semantic associations more visible. The data collection method in this paper still has shortcomings, and the collection process relies more on the rule information of commands. In addition, the generation process uses the concept of sequence-to-sequence, but the same AOM statement may have multiple equivalent natural language command sequences, so training a sequence-to-sequence model is prone to the “order problem”, and we will consider using reinforcement learning methods to overcome this problem. In conclusion, our approach provides a new way of thinking about natural human–machine interaction interfaces in agriculture, and we will further explore new areas of human–machine interactions in the future.

Author Contributions: Conceptualization, Y.Z. (Yongheng Zhang) and H.W.; methodology, Y.Z. (Yongheng Zhang); software, P.W.; validation, S.Y. and Z.X.; resources, Y.W.; data curation, P.W. and Z.X.; writing—original draft preparation, Y.Z. (Yongheng Zhang); writing—review and editing, Y.Z. (Yongheng Zhang) and S.Y.; project administration, Y.W.; funding acquisition, Y.Z. (Yongheng Zhang). All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by Anhui agricultural ecological and environmental protection and quality safety industrial technology system grant number 22803029.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Not applicable.

Acknowledgments: We sincerely thank the anonymous reviewers for their hard work. This research was completed with support from the Beidou Precision Agricultural Information Engineering Laboratory of Anhui Province. We thank them for providing technical support on modern agricultural measurement and control systems.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Razali, M.H.H.; Atar, M.; Norjihah, S. Human computer interaction and application in agriculture for education. *Int. J. Inf. Technol. Comput. Sci.* **2012**, *2*, 7–11.
2. Jansen, B.J. The graphical user interface. *ACM SIGCHI Bull.* **1998**, *30*, 22–26. [[CrossRef](#)]
3. Petersen, N.; Stricker, D. Continuous natural user interface: Reducing the gap between real and digital world. In Proceedings of the ISMAR, Orlando, FL, USA, 19–22 October 2009; pp. 23–26.
4. Alonso, G.; Casati, F.; Kuno, H.; Machiraju, V. Web services. In *Web Services*; Springer: Berlin/Heidelberg, Germany, 2004; pp. 123–149.
5. Bouguettaya, A.; Sheng, Q.Z.; Daniel, F. *Web Services Foundations*; Springer: Berlin/Heidelberg, Germany, 2014.
6. Bellegarda, J.R. Spoken language understanding for natural interaction: The siri experience. In *Natural Interaction with Robots, Knowbots and Smartphones*; Springer: New York, NY, USA, 2014; pp. 3–14.
7. Sarikaya, R.; Crook, P.A.; Marin, A.; Jeong, M.; Robichaud, J.P.; Celikyilmaz, A.; Kim, Y.B.; Rochette, A.; Khan, O.Z.; Liu, X.; et al. An overview of end-to-end language understanding and dialog management for personal digital assistants. In Proceedings of the 2016 IEEE Spoken Language Technology Workshop (SLT), San Diego, CA, USA, 13–16 December 2016; pp. 391–397.
8. Xu, X.; Liu, C.; Song, D. Ssqlnet: Generating structured queries from natural language without reinforcement learning. *arXiv* **2017**, arXiv:1711.04436.
9. Su, Y.; Awadallah, A.H.; Khabza, M.; Pantel, P.; Gamon, M.; Encarnacion, M. Building natural language interfaces to web apis. In Proceedings of the 2017 ACM Conference on Information and Knowledge Management, Singapore, 6–10 November 2017; pp. 177–186.
10. Austerjost, J.; Porr, M.; Riedel, N.; Geier, D.; Becker, T.; Scheper, T.; Marquard, D.; Lindner, P.; Beutel, S. Introducing a Virtual Assistant to the Lab: A Voice User Interface for the Intuitive Control of Laboratory Instruments. *SLAS Technol. Transl. Life Sci. Innov.* **2018**, *23*, 476–482. [[CrossRef](#)] [[PubMed](#)]
11. Mehrabani, M.; Bangalore, S.; Stern, B. Personalized speech recognition for Internet of Things. In Proceedings of the 2015 IEEE 2nd World Forum on Internet of Things (WF-IoT), Milan, Italy, 14–16 December 2015.
12. Noura, M.; Heil, S.; Gaedke, M. Natural language goal understanding for smart home environments. In Proceedings of the 10th International Conference on the Internet of Things, Malmö Sweden, 6–9 October 2020.
13. Feng, J. A general framework for building natural language understanding modules in voice search. In Proceedings of the 2010 IEEE International Conference on Acoustics, Speech and Signal Processing, Dallas, TX, USA, 14–19 March 2010; pp. 5362–5365.

14. Maynard, D.; Bontcheva, K.; Augenstein, I. *Natural Language Processing for the Semantic Web*; Synthesis Lectures on Data, Semantics, and Knowledge; Springer: Cham, Switzerland, 2016; Volume 6, pp. 1–194.
15. Dar, H.S.; Lali, M.I.; Din, M.U.; Malik, K.M.; Bukhari, S.A.C. Frameworks for querying databases using natural language: A literature review. *arXiv* **2019**, arXiv:1909.01822.
16. Jin, H.; Luo, Y.; Gao, C.; Tang, X.; Yuan, P. ComQA: Question answering over knowledge base via semantic matching. *IEEE Access* **2019**, *7*, 75235–75246. [[CrossRef](#)]
17. Ait-Mlouk, A.; Jiang, L. KBot: A Knowledge graph based chatBot for natural language understanding over linked data. *IEEE Access* **2020**, *8*, 149220–149230. [[CrossRef](#)]
18. Song, J.; Liu, F.; Ding, K.; Du, K.; Zhang, X. Semantic comprehension of questions in Q&A system for Chinese Language based on semantic element combination. *IEEE Access* **2020**, *8*, 102971–102981.
19. Xu, K.; Reddy, S.; Feng, Y.; Huang, S.; Zhao, D. Question answering on freebase via relation extraction and textual evidence. *arXiv* **2016**, arXiv:1603.00957.
20. Chu, E.T.H.; Huang, Z.Z. DBOS: A dialog-based object query system for hospital nurses. *Sensors* **2020**, *20*, 6639. [[CrossRef](#)] [[PubMed](#)]
21. Tian, Y.; Shou, L.-D.; Chen, K.; Luo, X.-Y.; Chen, G. Natural Language Interface for Databases with Content-based Table Column Embeddings. *Comput. Sci.* **2020**, *47*, 60–66.
22. V.S., A.; Kamath S, S. Discovering composable web services using functional semantics and service dependencies based on natural language requests. *Inf. Syst. Front.* **2019**, *21*, 175–189. [[CrossRef](#)]
23. Qiu, M.; Li, F.L.; Wang, S.; Gao, X.; Chen, Y.; Zhao, W.; Chen, H.; Huang, J.; Chu, W. Alime chat: A sequence to sequence and rerank based chatbot engine. In Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers), Vancouver, LM, Canada, 30 July–4 August 2017; pp. 498–503.
24. Wu, H.; Shen, C.; He, Z.; Wang, Y.; Xu, X. SCADA-NLI: A Natural Language Query and Control Interface for Distributed Systems. *IEEE Access* **2021**, *9*, 78108–78127. [[CrossRef](#)]
25. Qin, L.; Xie, T.; Che, W.; Liu, T. A Survey on Spoken Language Understanding: Recent Advances and New Frontiers. In Proceedings of the International Joint Conference on Artificial Intelligence, Montreal, QC, Canada, 19–27 August 2021.
26. Sutskever, I.; Vinyals, O.; Le, Q.V. Sequence to sequence learning with neural networks. In Proceedings of the Advances in Neural Information Processing Systems 27, Montreal, QC, Canada, 8–13 December, 2014.
27. Mikolov, T.; Karafiát, M.; Burget, L.; Cernocký, J.; Khudanpur, S. Recurrent neural network based language model. In Proceedings of the ‘Eleventh Annual Conference of the International Speech Communication Association’, Chiba, Japan, 26–30 September 2010.
28. Sundermeyer, M.; Schlüter, R.; Ney, H. LSTM neural networks for language processing. In Proceedings of the Interspeech, Portland, OR, USA, 9–13 September 2012; Volume 2012, pp. 194–197.
29. Dey, R.; Salem, F.M. Gate-variants of gated recurrent unit (GRU) neural networks. In Proceedings of the 2017 IEEE 60th international midwest symposium on circuits and systems (MWSCAS), Boston, MA, USA, 6–9 August 2017; pp. 1597–1600.
30. Huang, Y.; Jiang, Y.; Hasan, T.; Jiang, Q.; Li, C. A topic BiLSTM model for sentiment classification. In Proceedings of the 2nd International Conference on Innovation in Artificial Intelligence, Shanghai China, 9–12 March 2018; pp. 143–147.
31. Zhou, L.; Bian, X. Improved text sentiment classification method based on BiGRU-Attention. *J. Phys. Conf. Ser.* **2019**, *1345*, 032097. [[CrossRef](#)]
32. Devlin, J.; Chang, M.W.; Lee, K.; Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv* **2018**, arXiv:1810.04805.
33. Chorowski, J.; Jaitly, N. Towards better decoding and language model integration in sequence to sequence models. *arXiv* **2016**, arXiv:1612.02695.
34. Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A.N.; Kaiser, Ł.; Polosukhin, I. Attention is all you need. In Proceedings of the Advances in Neural Information Processing Systems 30, Long Beach, CA, USA, 4–9 December 2017; Volume 30.
35. Shaw, P.; Uszkoreit, J.; Vaswani, A. Self-attention with relative position representations. *arXiv* **2018**, arXiv:1803.02155.